

FPGA implementation of SRAM PUFs based cryptographically secure pseudo-random number generator

Shuai Chen*, Bing Li*, Cenjun Zhou

Advanced Cloud-System Research Center, School of Integrated Circuit, Southeast University, Nanjing 210000, China

ARTICLE INFO

Keywords:

CSPRNG
SRAM PUFs
Entropy extraction
FPGA

ABSTRACT

The security of cryptographic protocols depends on the security of key sequences consisting of random numbers. In this paper, we design a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG), which consists of a hash based Deterministic Random Bit Generator (DRBG) and a Get Entropy module. SRAM Physical Unclonable Functions (PUFs) are regarded as entropy sources, providing entropy data with enough entropy for CSPRNG. The construction of Get Entropy module is proposed to verify the availability of SRAM PUFs and compress the entropy data into truly random seeds that are fed into DRBG. This CSPRNG can reseed itself dynamically and can monitor the entropy of entropy sources in real time. The system is implemented and tested on Altera DE2. The test results show that, the pseudo-random numbers generated by this system can pass all random tests of National Institute of Standards and Technology (NIST) SP800-22 Test Suite and the throughput is up to 598.1 Mbps. Through the security discussion, this CSPRNG is theoretically confirmed that it can be securely applied to cryptographic protocols.

1. Introduction

Random Number Generators (RNGs) are crucial cryptographic primitives in encryption systems. They are widely used in key generation and authentication protocols. The security of these systems is built on such an assumption: the future values of random sequences cannot be predicted from the observed sequences [1]. True Random Number Generator (TRNG) and Pseudo-Random Number Generator (PRNG) are normally combined to be applied in encryption systems. The PRNG, which can generate pseudorandom bits deterministically by a deterministic algorithm and a random seed, are widely used in the scenario that need large amount of random bits. TRNG, compared to PRNG, is specifically based on a physical unpredictable attribute, such as metastable or thermal noise, and can be applied to the scenarios that have relatively small quantity demands of random bits. The short output of TRNG can be derived from a physical noise source whereas a PRNG expands the short output into a long sequence of seemingly random bits based on a deterministic algorithm. In PRNG, the initial seed, namely, the output of TRNG, is a key and necessary condition to generate pseudo-random sequences. The randomness of initial seed is also an important indicator to measure the performance of PRNG. Therefore, the seed must be confidential and shall not be predicted by attackers.

The security of cryptographic techniques directly depends on the

strength of secret keys. Traditionally, the key storage scheme relies on Non-volatile Memory (NVM) to encapsulate the secret keys in IC, e.g., EEPROM, FLASH and SRAM with uninterrupted power supply. Nevertheless, EEPROM and Flash-based storage scheme require adding floating gate transistor in the traditional CMOS process. SRAM-chip must be equipped with uninterrupted power, increasing the power consumption. Furthermore, NVM-based storage system is vulnerable to physical attacks. The proposed Physical Unclonable Functions (PUFs) provides the possibility for low-cost key storage scheme. In the PUFs-based security scheme, no secret needs to be securely stored. Instead, secrets are derived from the physical properties of the PUF when needed [2]. Therefore, PUFs gradually become an indispensable primitive in hardware security.

SRAM PUFs is one of the most popular PUFs implementation, because 1) they are widely used in existing systems ranging from sensors to microcontrollers and 2) the measuring circuits that extract intrinsic features from SRAM PUFs are very simple and without any design constraints (compared to the delay measuring circuits like Arbiter PUFs). We can obtain random information on SRAM start-up patterns. Therefore, serving SRAM PUFs as seed sources that provide initial seeds to PRNG is feasible. However, not all of the SRAM chips can be used as SRAM PUFs due to the variation of entropy (see Sections 2 and 3). Therefore, we propose a Get Entropy module to check the entropy in SRAM start-up patterns online. In this paper we design the construction

* Corresponding author.

E-mail addresses: chenshuai_ic@seu.edu.cn (S. Chen), bernie_seu@seu.edu.cn (B. Li), zhoucenjun@msn.cn (C. Zhou).

of a streaming random number generator according to the specification of NIST SP800-90A [3], which utilizes the full entropy seeds to generate pseudo-random numbers.

Furthermore, it should be mentioned that in the area of information security, “secret” is a relative concept. Generally speaking, it is the security of the system under certain attacks. For example, PUFs-based key storage scheme is more secure than NVM because PUFs can resist the physical attacks.

1.1. Related work

Since the introduction of optical PUFs in 2001 [4], researchers have proposed many different PUF designs. The most widely used ones are electronic PUFs. Delay-based PUFs are based on race conditions in ICs and include Arbiter PUFs [5,6] and Ring Oscillator (RO) PUFs [7,8]. To implement this kind of PUFs one needs to design two same signal paths that insure the same signal delay of them. However, due to manufacturing process variations, there is a slightly difference between signal delay of both signal paths and this difference can be used to generate secrets. Another kind of PUFs, Memory-based PUFs, is based on instability of volatile memory elements and includes SRAM PUFs [9], Flip-Flop PUFs [10] and Latch PUFs [11]. Unlike Delay-based PUFs, Memory-based PUFs are unstable circuits and the states of cells in these PUFs would vibrate if there is no initial input during powering up. This vibration can be used to generate secrets.

Finally, several papers have been written about using PUFs, especially SRAM PUFs, for random number generation. The paper [12] focused on the instability extracted from the start-up patterns of SRAM chips and proposed the possibility of using the instability to generate random numbers. Based on the paper [12], the possibility was proven and a preliminary construction was proposed in [13,14] constructed a lightweight PRNG with SRAM PUFs, but the lack of reseeding operation cannot insure the security of the PRNG.

1.2. Our contribution

The novelty of our work is twofold:

- 1) Monitoring the entropy of seed extracted from SRAM PUFs in real-time. The characteristics of seed provide the basis for the security of DRBG. The seed in DRBG needs to meet two conditions: security and contain at least the requested amount of entropy. In previous work, the problem of whether or not the SRAM PUFs chips can meet the entropy requirements was based on a large number of experimental data obtained in advance [12–14]. However, as shown in Section 2.1, the entropy of SRAM PUFs was unstable when using in-the-field due to the effects of noise or other elements. This will affect the security of the system. Therefore, in view of this vulnerability, we designed a Get Entropy module in our proposal, which can collect noises from SRAM start-up patterns and calculate the entropy contained in noises real-time. The construction we propose is applicable to a wide range of SRAM chips and Sections 2 and 3 contain more details on our construction.
- 2) Period Reseeding. The DRBGs recommendation [1] proposed that periodic reseeding is a good way of addressing the threat of the DRBG seed, entropy input or working state becoming compromised over time. Therefore, compared to the existing work, in this paper, we proposed a period reseeding DRBG scheme combined with the experiment results in Section 3.1 (these tests is also the first to concerned the relationship between data remanence in SRAM chips and the entropy). Furthermore, the Get Entropy module aforementioned will be used in this reseeding process to ensure the appropriate entropy input for the request security length.

Furthermore, most cryptosystems use linear feedback shift register (LFSR) or modified LFSR to generate pseudo-random numbers. Its linear

behavior allows an encryption key to be easily recovered or stolen [15]. The simple topologic make LFSR unsafe in the high security application due to the reason that the topologic can be speculated by some attack (e.g., modeling attack) and then the next bit stream will be predicted, when the $2n$ (n is number of stage) bits are obtained. Indeed, the LFSR-based Stream-Ciphers have relatively simple hardware circuitry and are faster and consume very low power. Therefore, it is widely used in some resource-constrained scenario, e.g., RFID. However, paper [16] presents an attack for this scheme by determining the scan chain structure of the LFSR in the implementation. Several attacks that target simple but general LFSR structures are described and follow it up with demonstrating these attacks on six LFSR-based stream ciphers. So an attacker can duplicate an authentication device, eavesdrop a private conversation, if he successfully attacks the LFSRs used in stream cipher devices.

Therefore, we design integrated hardware architecture of DRBG according to the specification of NIST SP800-90A. This DRBG takes the Hash algorithm as core algorithm and can reseed itself dynamically that insures backtracking resistance and prediction resistance. These resistances improve the security of CSPRNG.

1.3. Paper organization

The remainder of this paper is organized as follows: Section 2 introduces the concept of SRAM PUFs and focuses on the theory of extracting entropy. Section 3 presents the construction of Get Entropy module and gives the test results of key parameters in the module. Section 4 describes the construction of DRBG. The FPGA implementation and the results of NIST tests are presented in Section 5. The discussion on the security of PRNG system is presented in Section 6. Finally, the conclusions of this publication are gathered in Section 7.

2. Basis of SRAM PUFs

2.1. Structure of SRAM

Each bit of SRAM is a six-transistor memory cell that consists of cross-coupled CMOS inverters and access transistors [17]. The two symmetrical inverters (labeled “A” and “B” in Fig. 1) can keep SRAM cells in two states corresponding to logical 0 and 1. For a SRAM PUFs cell, the manufacturing variations are frozen after manufacturing process. Hence this manufacturing mismatch is stable and provides the noise margin of the cell. When this cell is used in the field, if the influence of noise (temperature, voltage) is insufficient to cover the noise margin, the output will be stable (0 or 1) and named 0-skewed or 1-skewed cells in this paper. Else, the cell is unstable with relatively better matching (close to equilibrium with small noisy margin). So, both stable and unstable cells are the reflection of manufacturing variation, and can be seen as the IC fingerprint. The three types of SRAM cells are randomly distributed over the array of memory cells that form the SRAM block. The main targets of our design are to eliminate 0-skewed

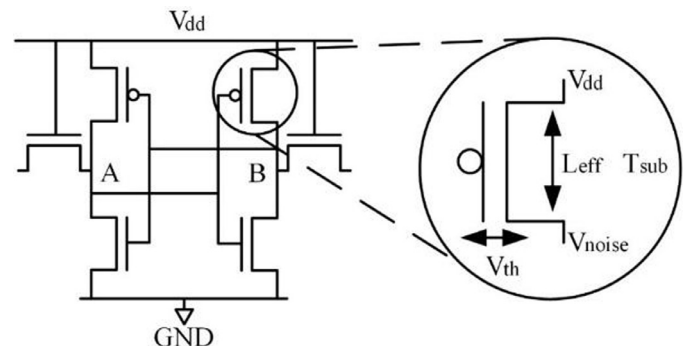


Fig. 1. Structure of SRAM cell.

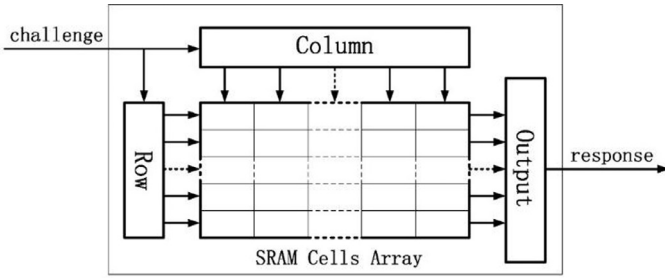


Fig. 2. Structure of SRAM PUF.

and 1-skewed cells and utilize neutral-skewed cells to generate entropy data.

2.2. Principles of entropy extraction

The basic structure of SRAM PUF is shown in Fig. 2. As a kind of weak PUFs, the challenge to an SRAM PUF is a SRAM address after powering up, while the corresponding response is the content of the uninitialized SRAM cells at this address. Therefore, entropy data can be extracted from the PUF responses by constantly powering up and powering down the SRAM chip.

Since the DRBG algorithm itself is deterministic, the output only looks random to someone without knowledge of the seed value [18] and there will be an inevitable repetition of outputs after enough time has passed. Therefore, the security of DRBG depends on the security of input seeds to a great extent. These seeds must be unpredictable and computationally infeasible. In this paper, entropy is used to describe the security of input seeds. Entropy is defined relative to one's knowledge of a random variable prior to an observation, and reflects the uncertainty associated with predicting its value – the larger the entropy, the greater the uncertainty in predicting the value of an observation [19].

Min-entropy is often used as a worst-case measure of randomness for a random variable, which means the unpredictability of a random variable under the minimum chaos. In this design, as long as the min-entropy of entropy data provided by a SRAM PUF can be greater than the security strength required by DRBG, then this SRAM PUF can be used as a seed source of DRBG.

For a binary random variable, let P_0 and P_1 be the probability when the variable values are logical 0 and 1 respectively. Let P_{max} be the maximum value of P_0 and P_1 . The min-entropy is defined by:

$$H_{min} = -\log_2 P_{max} \quad (1)$$

Assuming that all cells in SRAM PUFs are independent, each cell can be viewed as an individual binary random variable. For n cells in SRAM PUFs, the following definition holds:

$$(H_{min})_{total} = -\sum_{i=1}^n \log_2 (P_i)_{max} \quad (2)$$

Hence, we can simply sum the min-entropy of each SRAM cell at the same time and calculate the total min-entropy in a SRAM PUF. This method can be efficiently implemented in FPGA.

3. Design of get entropy module

3.1. Functions

According to the Eq. (2) in Section 2.2, we design the Get Entropy module. The main functions of this module are shown as follows:

1. Verify whether or not the SRAM PUF can generate entropy data with enough entropy.
2. Obtain the number of SRAM cells that can generate entropy data as needed, which is stored in the form of SRAM address number.

3. Obtain entropy data from SRAM PUF according to the above address number.
4. Compress entropy data into seeds.

This module is divided into two stages at runtime. The first stage is testing stage that includes the function 1 and 2, and the second stage is compression stage that includes the function 3 and 4. Note that in order to prevent the entropy of SRAM PUF decreasing and even being failure caused by environmental temperature change or other noise changes, this module will return to the first stage to re-verify SRAM PUF after producing 100 seeds.

3.2. Parameters

In Get Entropy module, there are two key parameters: 1) Power down hold time – T ; 2) Number of measurements for entropy – N_{on-off} .

3.2.1. Power-down hold time – T

The first parameter T is related to the mechanism of data remanence in SRAM chips. The static RAM devices will retain data once the power has been removed. If the data retention time exceeds the time required by Get Entropy module to open the device and power up the memory, the state of SRAM cells will be influenced by the retention data and will not be metastable, which means the SRAM chip only can provide little entropy. This will lead to entropy insufficiency in seeds and less security of DRBG.

The experiment results based on [20] show that the data retention times of all SRAM chips used in the experiment are less than 3.5 s and can be significantly reduced by shorting VCC to ground rather than by leaving it floating. To be able to set up a suitable T that is specific to our Get Entropy module, measurements are performed on four different kinds of SRAM chips with five samples each at room temperature as shown in Fig. 3. The four kinds of SRAM chips are HM62256ALP-10 (HM1 ~ HM5), HY62256ALP-70 (HY1 ~ HY5), WS62256LLPG-70 (WS1 ~ WS5) and UT62256CPC-70LL (UT1 ~ UT5). We run the Get Entropy module in first stage until the led that indicates the success of verification is turned on. And we obtain the number of SRAM cells through a logic analyzer at this point. If we set a suitable T , this number will be relatively stable in several experiments.

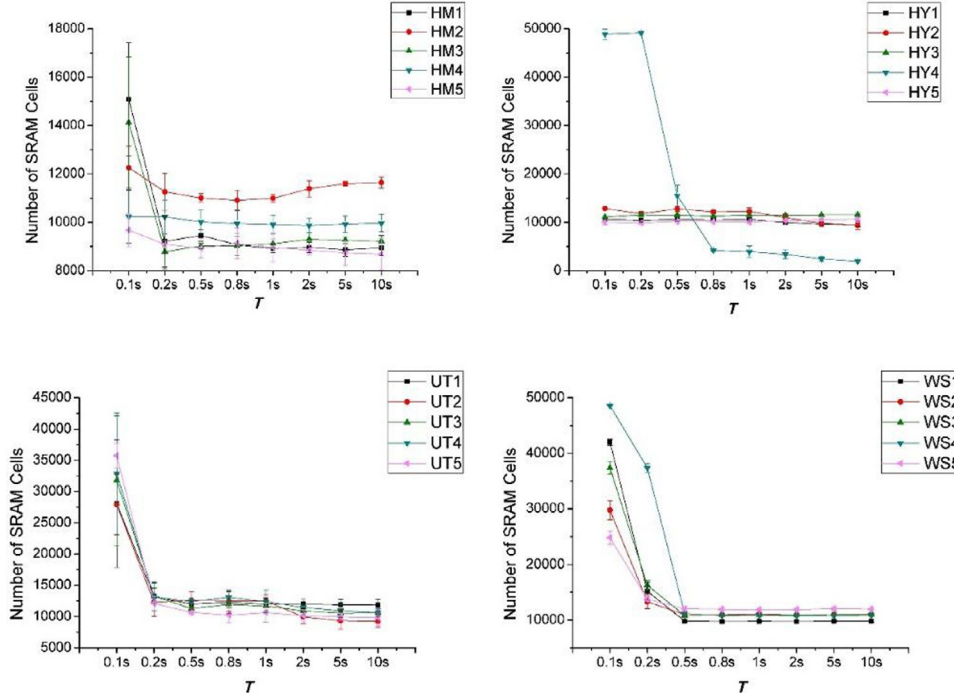
The points in the figure represent the averages and the T-shaped lines beside the points represent the fluctuating values. In Fig. 3, the differences between SRAM chips are mainly caused by different manufacturing processes and technology. However, this is not the focus of this paper. Our main point is to draw the following conclusions through the experiment results:

1. The number of SRAM cells that can produce required entropy tends to be stable when T is more than 5 s. This conclusion is based on the asymptotic behavior of the lines in the figure. Therefore, the parameter T in our design will be 5 s.
2. All of the SRAM chips used in measurements can conform to the requirements of our design. And even the same series of chips can have hugely different features.

3.2.2. Number of measurements for entropy – N_{on-off}

The second parameter N_{on-off} is related to the stability of entropy provided by SRAM cells. According to the Eq. (2), min-entropy is accumulated by collected the probability P_{0-1} which is got by summing the P_{max} in a certain SRAM cell. Only when the two probabilities P_0 and P_1 tend to be stable, the min-entropy conforms to the actual situation.

$$P_{0-1} = \sum_{j=1}^{N_{on-off}} (P_{max})_j / N_{on-off} \quad (3)$$

Fig. 3. Results of measurements on all chips with different T .

$$(H_{min})_{total} = - \sum_{i=1}^n \log_2(P_{0-1})_i \quad (4)$$

If the number of measurements N_{on-off} is too small, the imprecise min-entropy will lead to a decline in the safety of DRBG; whereas if N_{on-off} is too large, the collection time will be extended and a large number of logical resources will be wasted.

According to the experiment results in [13], when N_{on-off} is more than 50, the min-entropy of SRAM cells will tend to be stable. And when it is equal to 100, the min-entropy is nearly invariable. The results are chip-type-independent. Therefore, N_{on-off} in our Get Entropy module will be 100 that suffice to get an accurate estimation of the min-entropy in SRAM start-up patterns.

3.3. Construction

The proposed construction of Get Entropy module is depicted in Fig. 4 and consists of following components: Switch module, PUF initial module, PUF process module, counter module, SRAM control module, SHA-256 and control module, get module and a pipeline adder.

In Get Entropy module, testing stage is the process to verify whether

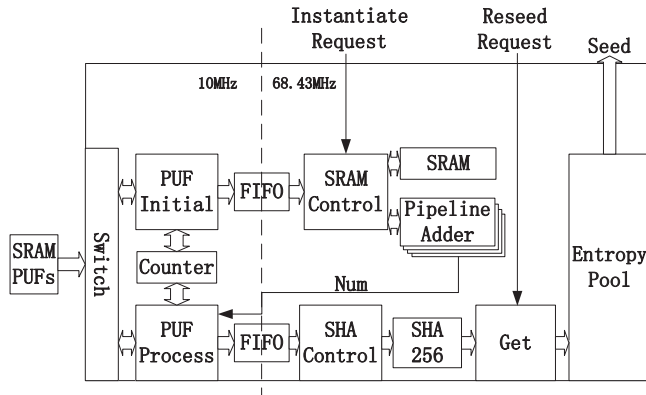


Fig. 4. Construction of get entropy module.

or not the SRAM PUFs can generate enough entropy data and obtain the number of SRAM cells that can generate entropy data as needed and stored it in the form of SRAM address number. Compression stage is to obtain entropy data from SRAM PUFs according to the above address number and compress this entropy data into 256-bits seed using hash algorithm (SHA-256).

Hash algorithm is a mathematical function that maps values from a large (possibly very large) domain into a smaller range. SHA-256 is a cryptographic hash functions designed by NIST [21]. In this module, SHA-256 is used to compress the large amount of SRAM PUFs output that have already meet the entropy requirements (256-bits min-entropy) in testing stage into a 256-bits seed that meet the entropy requirements and the seed length of DRBG. The rounds of this SHA-256 are following:

The procedure consists of two stages, namely, preprocessing and hash computation. In the preprocessing process, the input message is padded with one “1” and leading “0”s until the length of this message becomes a multiple of 512-bits. In hash computation process, a common sequence of operations is called a digest round and the compression function produces a hash value by subjecting a block of message to 64 digest rounds. In this paper, our SHA-256 cores are based on the SHA cores in Opencores [22].

3.3.1. Switch module

This component is used to switch the interface between PUF initial module, PUF process module and SRAM PUFs according to the running stages of Get Entropy module. When in testing stage, SRAM PUFs will be connected to PUF initial module. When in compression stage, SRAM PUFs will be connected to PUF Process module.

3.3.2. Counter module

This component controls power-up and power-down of SRAM chips and is controlled by PUF initial module and PUF process module. When these two modules issue instructions of powering down, the counter module will short SRAM VCC to ground and time the power-down time. After reaching T seconds, this component will power up SRAM chips again.

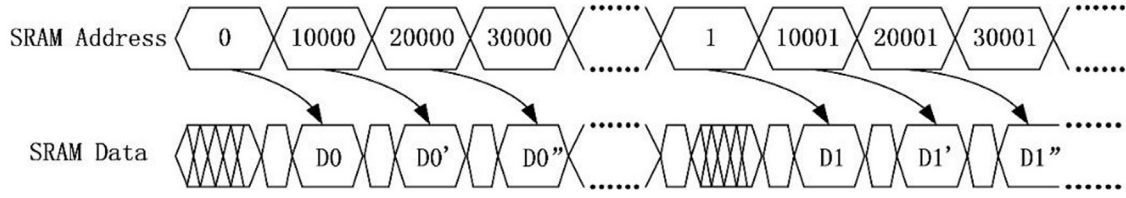


Fig. 5. Address operation rule.

3.3.3. PUF initial module

This component traverses addresses of SRAM PUFs to read out the uninitialized values after the process of powering down and powering up again. This operation will repeat N_{on-off} times and then these values will be written into FIFO in turn.

3.3.4. SRAM control module

This component reads out the values from FIFO and then writes them into the embedded SRAM in FPGA. After writing all values, which are $N_{on-off} \times Cells_{SRAM PUFs}$ bits ($Cells_{SRAM PUFs}$ is the number of cells in SRAM PUFs), SRAM control module will wait for the security strength request of the Instantiate module in DRBG. The value $(H_{min})_{total}$ that needs to be acquired is equal to the value of security strength. After receiving the request, this component will send the value $(H_{min})_{total}$ to pipeline adder, and input the data stored in embedded SRAM into pipeline adder according to certain address rule. This rule is shown in Fig. 5.

In our design, $Cells_{SRAM PUFs}$ is $32K \times 8$ bit and $Cells_{em-SRAM}$ in FPGA is $1M \times 16$ bit. Therefore, only the first 2×10^4 addresses of SRAM PUFs are verified. Because of the different bit widths in these two kinds of SRAM chips, the uninitialized value i and value $i + 1$ are stored in embedded SRAM at 10^4 addresses intervals. In order to sum up them, after reading the value of address 0, SRAM Control module should read the value of address 10,000. The address rule can be deduced by analogy. The Maximum entropy input length is 10,000 SRAM address ($10,000 \times 8$ -bits).

3.3.5. Pipeline adder

As shown in Fig. 6, the bit wide of the embedded SRAM in the FPGA development board (Altera DE2) is 16-bits. 16 SRAM PUFs cells' 100 times power up state can be stored in this 16×100 -bits SRAM array. Therefore, in this 16 stages, each stage accumulates one SRAM PUFs cells' P_{0-1} . Through referring a $\log_2$ lookup table, the final outputs are

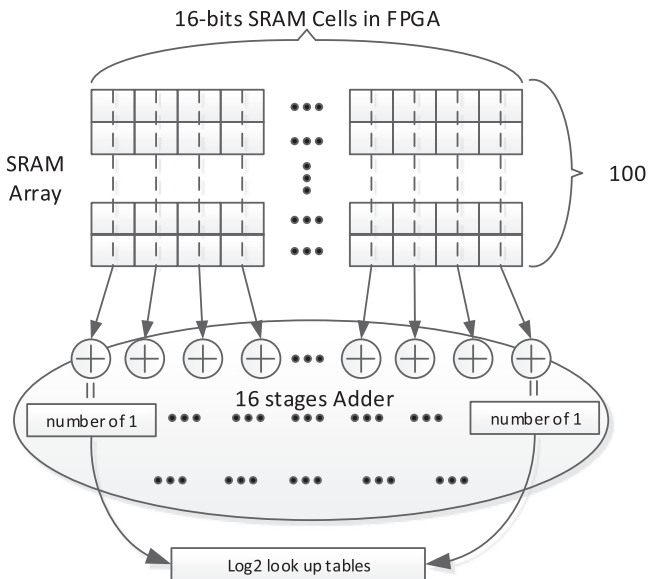


Fig. 6. 16 stages pipelines.

the sum of min-entropy of 16 cells (Eq. (3)). When this sum is larger than the security strength required by DRBG, this component will send the number of sets (Num) to PUF Process module. At this point, the first testing stage is successful that means current SRAM PUF can be used as seed source.

In this paper, we set $N_{on-off} = 100$ (as shown in Section 3.2), $P_{max} = \max(P_0, P_1) = \max(\text{number of 0, number of 1})/100$. Obviously, $\max(\text{number of 0, number of 1})$ is a constant range from 50 to 100. Therefore, the input of this tale is a 7-bits constant. For output of this $\log_2$ table, the accuracy of this output is designed to be 0.01. In operation, this output was expanded 100 times to avoid floating-point arithmetic. Therefore, the output is another 7-bits constant. In conclusion, the $\log_2$ look up table is consisted of 50 pairs of 7-bits input and 7-bits output.

3.3.6. PUF process module

This component can obtain entropy data in SRAM PUFs according to Num and compress the data into seeds through the hash algorithm. These seeds will be written into Entropy Pool waiting to be read. When the Entropy Pool is full, the process of producing seeds will stop. Note that the seeds in Entropy Pool will be emptied if Get Entropy Module returns the first stage.

4. Deterministic random number generator

4.1. Functions

The design of DRBG implements an approved DRBG mechanism according to the NIST SP 800–90A [3]. The main functions of this module are shown as follows:

1. Request a seed and generates an initial internal state.
2. Generate pseudo-random numbers according to the initial internal state and updates the internal state.
3. Request reseeding seed to regenerate internal state.

This Generator is divided into three stages at runtime: instantiate stage, generate stage, and reseed stage. Detailed running flow is shown in Fig. 7.

4.2. Parameters

In DRBG, there are four key parameters: (1) nonce; (2) personalization string; (3) additional input; (4) seed life. As shown in Fig. 7, in instantiation process (instantiate module), the initiate state was determined by seed and parameters (shown in Eq. (6)): (1) nonce; (2) personalization string. In generate process (generate module), the internal state was determined by initiate state and additional input (shown in Eq. (9)). In reseed process (reseed module), additional input was also one of the parameters to determine the internal state (shown in Eq. (13)).

A nonce is required in the construction of the initial internal state during instantiation in order to provide a security cushion to block certain attacks. This parameter can be composed of the following components: 1. A random value generated by an approved RNG; 2. A timestamp; 3. A monotonically increasing sequence number. In this

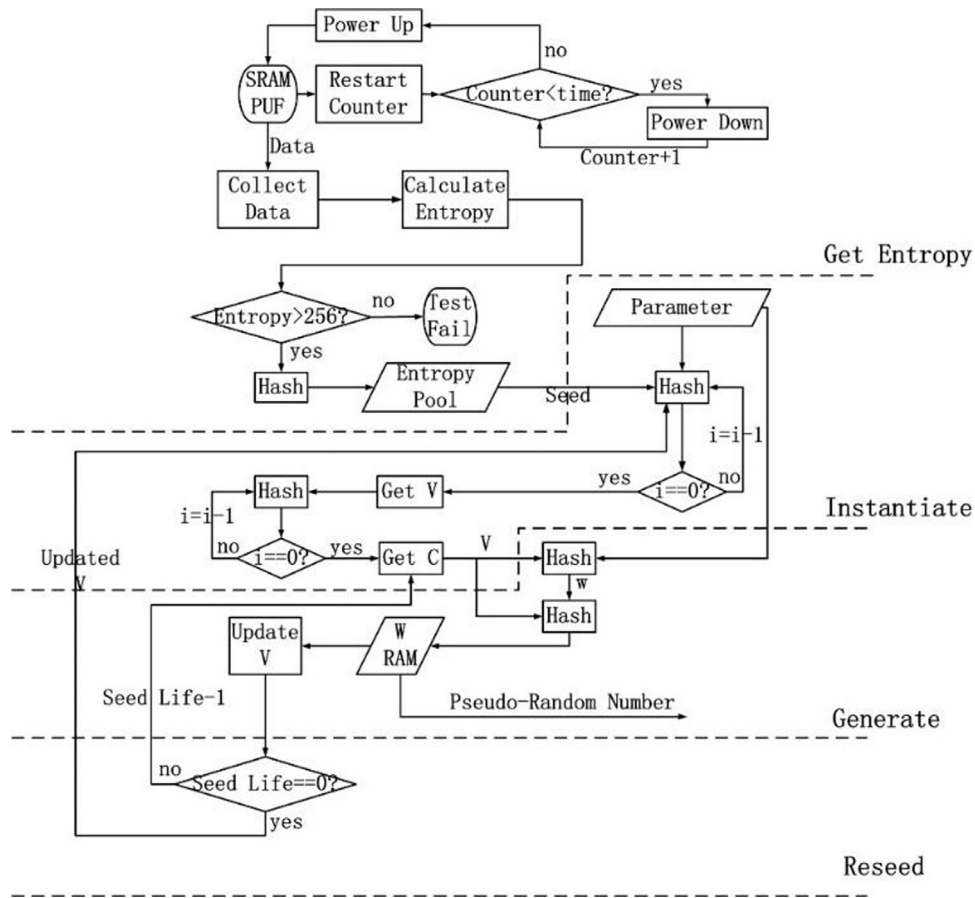


Fig. 7. DRBG running flow.

DRBG, nonce must be unique, but needs not be secret. Therefore, monotonically increasing sequence number is the simplest way to meet these requirements.

The personalization string should be unique for all instantiations of the same DRBG mechanism type. It does not provide entropy, even though entropy could be provided in the personalization string, and knowledge of the personalization string by an adversary does not degrade the security strength of a DRBG instantiation, as long as the entropy input is unknown. This personalization string can be composed of the following components: 1. Application identifiers; 2. Device serial numbers; 3. User identification. 4. Random numbers and so on. Therefore, we choose a random number sequence generated from a RNG in our computer (by Matlab) as the personalization string to replace the device serial numbers that exists in the devices that this DRBG may be used.

An additional input is optionally provided to generate and reseed module during requests. We choose another random number sequence to be the additional input.

Seed life is a parameter that indicates the number of times that a seed can be used to generate pseudo-random numbers. The internal counter in generate module will increase each time the pseudo-random numbers generated. When the count value of seed life is reached, generate module must stop generation process and start reseeding process to re-obtain seeds.

4.3. Constructions

The proposed construction of DRBG is depicted in Fig. 8 and consists of following modules: Data Control module, Instantiate module, Generate module, and Reseed module. In this section, all the constant values in the formulas, including decimal and hexadecimal values, are

chosen based on the NIST 800–90A.

4.3.1. Data control module

This module is the memory of DRBG and consists of all of the parameters, commands and other stored values that DRBG uses or acts upon.

4.3.2. Instantiate module

DRBG must be instantiated in advance of generating pseudo-random numbers and the instantiation process can only be performed once during the system operation. The instantiation process is shown as following:

1. Confirm the effectiveness of the input parameters.
2. Request a seed (256-bits) with entropy sufficient to support the required security strength.
3. Request the nonce and the personalization string.
4. Determine the initial internal state using Hash algorithm.
5. Send the initial internal state to RAM.

In instantiation module, referring to the SHA-256 based Hash_DRBG in NIST SP 800–90A, the seed material length (*seedlen*) is 440-bits, the output block length (*outlen*) is 256-bits. Therefore, the $len = seedlen / outlen = 0 \times 02$ (a 32-bit string). We defined $counter = 0 \times 01$ and $temp =$ The null string. The instantiation process **Hash_df** is:

For $i = 1$ to len do {temp

$$= temp || \text{Hash}(counter || len || seed\ material), counter = counter + 1 \}$$
(5)

The output of **Hash_df** = **leftmost**(temp, *seedlen*).

Therefore, the output of instantiation module (internal state) V_{ini}

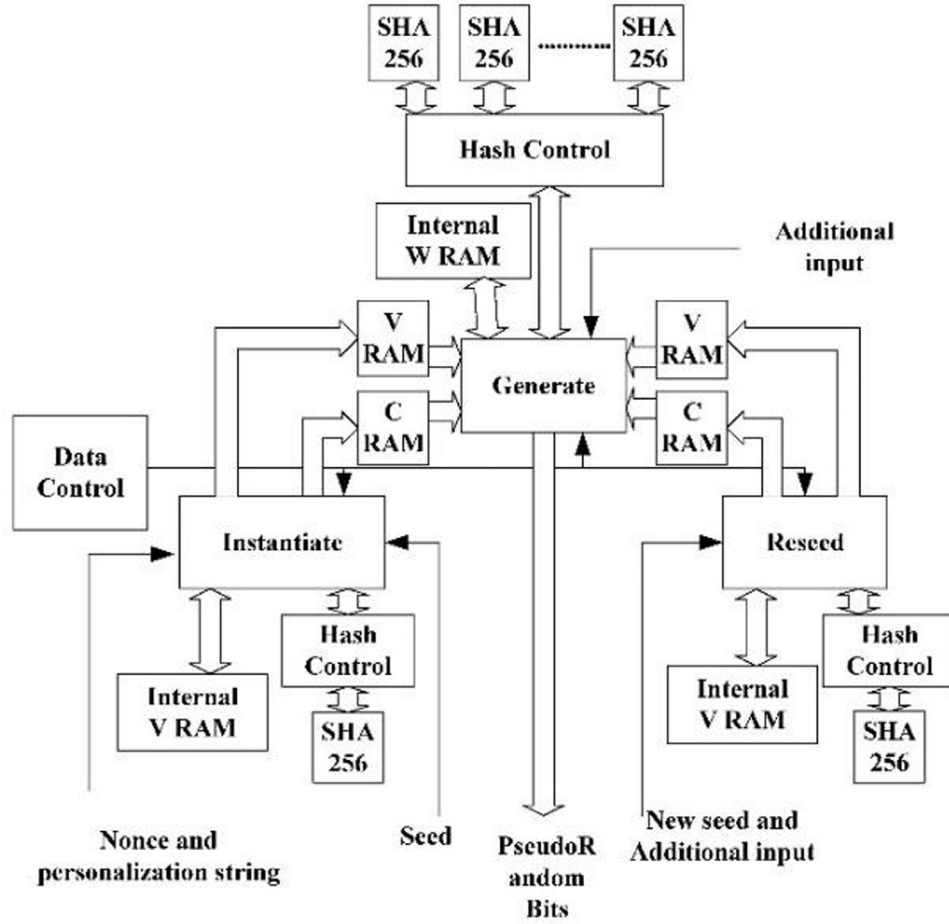


Fig. 8. Construction of DRBG.

and C_{ini} are following:

$$\text{Seed material} = \text{seed} \parallel \text{nonce} \parallel \text{personalization string} \quad (6)$$

$$V_{ini} = \text{Hashdf}(\text{seed material}, \text{seedlen}) = \text{leftmost}(\text{Hash}(0 \times 01 \parallel 32'h440 \parallel \text{seed material}) \parallel \text{Hash}(0 \times 02 \parallel 32'h440 \parallel \text{seed material}), 440) \quad (7)$$

$$C_{ini} = \text{Hashdf}((0 \times 00 \parallel V_{ini}), \text{seedlen}) = \text{leftmost}(\text{Hash}(0 \times 01 \parallel 32'h440 \parallel 0 \times 00 \parallel V_{ini}) \parallel \text{Hash}(0 \times 02 \parallel 32'h440 \parallel 0 \times 00 \parallel V_{ini}), 440) \quad (8)$$

Note that $\text{leftmost}(a, b)$ is the leftmost a bits of b .

4.3.3. Generate module

After instantiation or reseeding, Generate module is used to generate pseudo-random numbers utilizing the initial internal state or update internal state. This DRBG supports four security strengths: 112 bits, 128 bits, 192 bits and 256 bits. The maximum random output bit length in one request process is 2^{19} -bits. The generation process is shown as following:

1. Confirm the effectiveness of the input parameters.
2. Call reseed module for an update internal state if seed life reaches.
3. Request initial internal state and the additional input.
4. Generate a set of pseudo-random numbers using Hash algorithm.
5. Update the internal state.
6. Return the pseudo-random numbers to the application and returns to the first step.

In generate module, *returned_bits* is the output pseudorandom bits and *request_number* is the number of this pseudorandom bits. *Seed_counter* is a value to count the number of generation requests. If *seed_counter* > *seed life*, else, start the generate process:

$$V_{gen} = V_{ini} + \text{Hash}(0 \times 02 \parallel V_{ini} \parallel \text{additional input}) \quad (9)$$

Constant $m = \text{request_number} / \text{outlen}(256\text{-bits})$, therefore, the output pseudorandom bits is

$$\text{returnedbits} = \text{leftmost}(\text{Hash}(V_{gen}) \parallel \text{Hash}(V_{gen} + 1) \parallel \dots \parallel \text{Hash}(V_{gen} + m, \text{requestnumber})) \quad (10)$$

And then, refresh the internal state

$$V_{gen} = V_{gen} + \text{Hash}(0 \times 03 \parallel V_{gen}) + C_{ini} + \text{seedcounter} \quad (11)$$

$$\text{seedcounter} = \text{seedcounter} + 1 \quad (12)$$

Note that if Generate module calls reseed module, V_{ini} and C_{ini} should be replaced by V_{update} and C_{update} .

In order to implement Eq. (10) efficiently, our design adopts parallelized SHA-256 cores to make hash computation. These cores are controlled by a Hash Control module.

4.3.4. Reseed module

The process of reseeding shall be performed when Generate module generates overmuch output from one single seed or when the seed life reaches. Using one single seed for too many times may lead to the information leakage of DRBG, that is to say, the characteristic of prediction resistance of DRBG cannot be guaranteed. Therefore, the process of reseeding shall be performed when Generate module generates

overmuch output from one single seed or when the seed life reaches. This module can retain previous entropy in system through combining the current entropy input (may not have required entropy) with the last residual internal state. The maximum interval between reseeding is *seed life*. According to the recommendation in NIST SP 800–90A, *seed life* can be any constant from 1 to 2^{48} . In this paper, we set *seed life* = 5981. In one get entropy process, 100 seed will be stored in entropy pool. In generate process, 5981×100 *seed life* will costs about 500S. This is the time that get entropy module complete the next get entropy process. And then, there will be 100 new seed that can be used in reseeding process. There was almost no waiting time in periodic process because these two modules running in parallel.

This module can retain previous entropy in system through combining the current entropy input (may not have required entropy) with the last residual internal state. The reseeding process is shown as following:

1. Confirm the effectiveness of the input parameters.
2. Request a new seed.
3. Request current internal state and the additional input.
4. Determine the update internal state of DRBG using Hash algorithm.
5. Send the update internal state to RAM.

The process of reseed module is based on the Hash_df algorithm aforementioned in instantiation module. This module determines the update internal state V_{update} and C_{update} according to following Eqs. (13)–(15) and then *seed_counter* = 1:

$$\text{New seed material} = 0 \times 01 || V_{gen} || \text{new seed} || \text{additional input} \quad (13)$$

$$\begin{aligned} V_{update} = & \text{Hashdf}(\text{new seed material}, \text{seedlen}) = \text{leftmost} \\ & (\text{Hash}(0 \times 01 || 32'h440 || \text{seed material} || \text{Hash}(0 \\ & \times 02 || 32'h440 || \text{seed material}), 440) \end{aligned} \quad (14)$$

$$\begin{aligned} C_{update} = & \text{Hashdf}((0 \times 00 || V_{ini}), \text{seedlen}) = \text{leftmost}(\text{Hash}(0 \\ & \times 01 || 32'h440 || 0 \times 00 || V_{ini}) \\ & || \text{Hash}(0 \times 02 || 32'h440 || 0 \times 00 || V_{ini}), 440) \end{aligned} \quad (15)$$

5. Implementation of PRNG system and tests

5.1. FPGA implementation

The implementation of CSPRNG system targets the Altera Cyclone® IV 4CE115 FPGA device on the DE2-115 board (cf. Fig. 9). Because the automatic initialization of internal SRAM in FPGA, we must use additional external SRAM chip as SRAM PUF. This SRAM chip is carried on the minimum system board (labeled “2” in Fig. 9). This singlechip is connected to the GPIO of DE2-115 board through ribbon cables. The Counter module in Get Entropy module controls power-up and power-down of SRAM chips through the CS signal which is connected with a triode switching circuit (labeled “1” in Fig. 9).

The circuit structure of the switching circuit is shown in Fig. 10. The CS signal is connected to node 3, and the VCC of SRAM is connected to node 2. When the CS signal is logical 1, the voltage of node 2 is close to 0 V; whereas it is logical 0, the voltage is close to 5 V.

Adopting multiple parallelized SHA-256 cores in Generate module can greatly improve the speed of CSPRNG system, whereas it will consume more resources (as shown in Table 1). Therefore, the number of parallelized SHA-256 should be configured according to the scenario of it used. In this paper, we used 4 parallelized SHA-256 cores in our

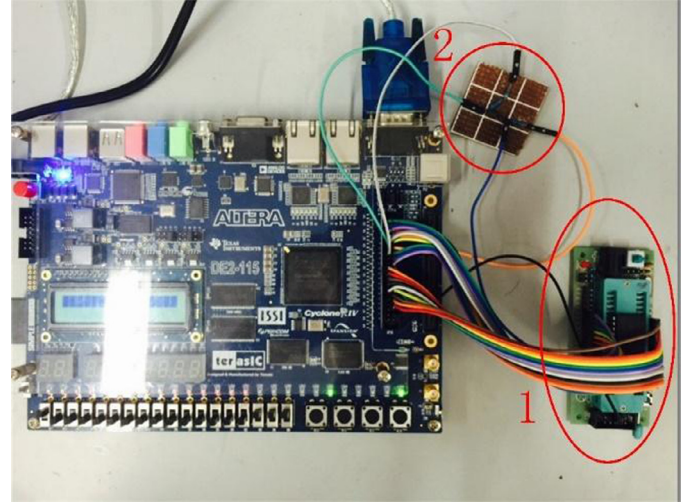


Fig. 9. FPGA implementation of PRNG system.

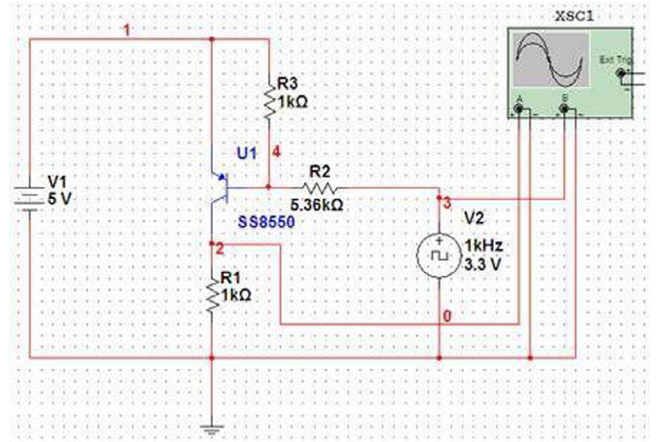


Fig. 10. Structure of switching circuit.

Table 1

Implementation compilation results in Altera Cyclone® IV 4CE115.

Get entropy module	Logic EL	Memory Bits	Throughput
	5049	303,104	
Configuration 1: parallelized SHA-256 cores = 2			
DRBG	15,822	15,488	
Total system	20,874	318,592	346.4Mbps
Configuration 2: parallelized SHA-256 cores = 4			
DRBG	20,694	15,488	
Total system	25,746	318,592	598.1Mbps
Configuration 3: parallelized SHA-256 cores = 8			
DRBG	30,438	15,488	
Total system	35,487	318,592	939.5Mbps

DRBG implementation to carry out the experiments.

5.2. Statistical randomness tests

This subsection describes the statistical randomness tests that are performed on the pseudo-random numbers generated by CSPRNG system. The test platform is shown in Fig. 11. In order to prove that this PRNG system is applicable to a wide range of SRAM chips, 100 measurements have been performed on each series of SRAM chips (HM62256ALP-10, HY62256ALP-70, UT62256CPC-70LL and WS62256LLPG-70). Each measurement contains 100 strings of length 1.003.520.

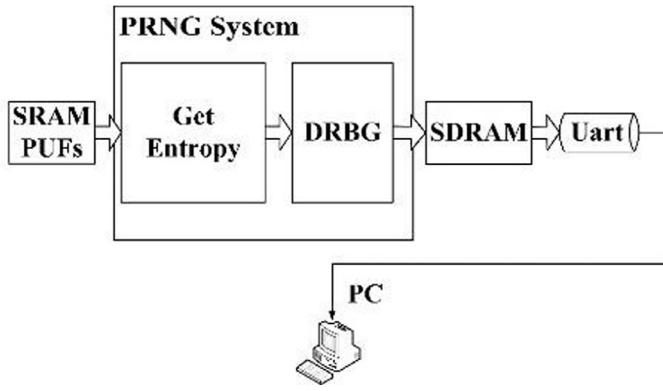


Fig. 11. Test platform for PRNG system.

In this paper, the NIST randomness test suite from the Special Publication 800–22 [23] has been used. This test suite is used as a first step to verify whether or not a generator is a CSPRNG. In this random number tests, the null hypothesis (H_0) is that the sequence being tested is random and the alternative hypothesis (H_A) is that the sequence is not random. A P -value is introduced to summarize the strength of the evidence against the null hypothesis. A significance level α is chosen to be compared with P -value. If P -value $\geq \alpha$, then the null hypothesis is accepted; i.e., the sequence appears to be random. If P -value $< \alpha$, then

Table 2

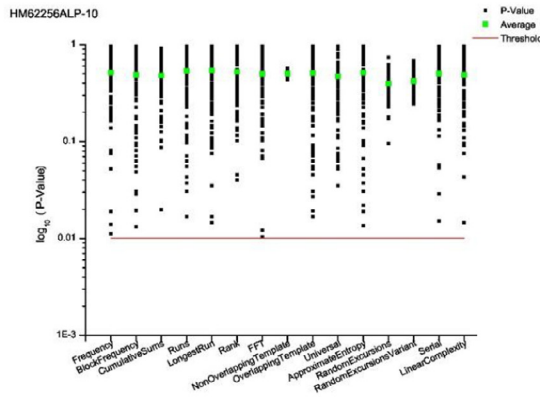
Comparisons of PRNGs in throughput, reseed operation, entropy real-time test and statistical properties proven.

PRNG	Th (Mbps)	Resources consumption	Reseed	Entropy test	Test SP 800–22
Hash_DRBG [13]	400	30 K Gates	×	Non-real time	Pass
RM-PRNG [24]	6400	11.9 K Gates	✓	N.A.	Pass
Rényi [25]	200	N.A.	×	N.A.	Pass
LGM [26]	200	9 K Gates	✓	N.A.	Fail
NLFSR [27]	400	2.5 K Gates	×	N.A.	N.A.
PUFKEK [14]	803	17.7 K Gates	×	Non-real time	Pass
CSPRNG System	598.1	158.3 K Gates	✓	Real time	Pass

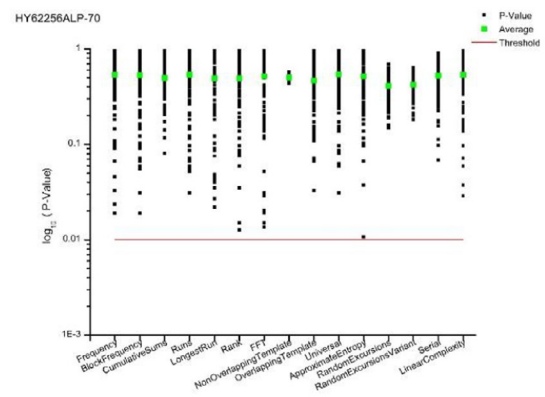
N.A. means “Not Available”.

the null hypothesis is rejected; i.e., the sequence appears to be non-random [23]. In this paper, we chose $\alpha = 0.01$, which means the sequence would be considered to be random with confidence of 99% when P -value ≥ 0.01 . The results are shown in Fig. 12. From the results can be seen that all strings pass the NIST randomness tests.

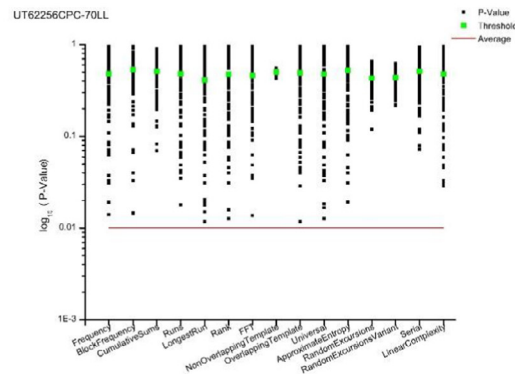
Table 2 tabulates the comparative results for Hash_DRBG [13], RM-PRNG [24], Rényi Map PRNG [25], LGM [26], NLFSR [27], PUFKEK [14] and our CSPRNG system in terms of throughput(= Mbps), reseed operation, entropy real-time test and statistical quality assessed by NIST



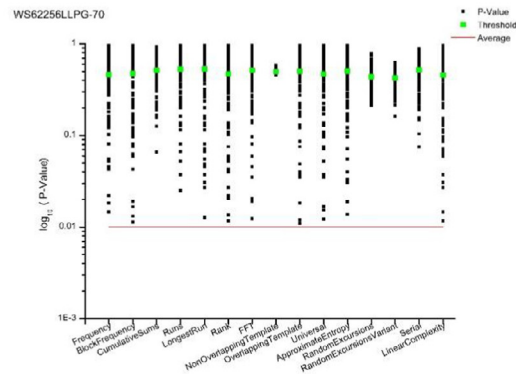
(a) Test results of HM62256ALP-10



(b) Test results of HY62256ALP-70



(c) Test results of UT62256CPC-70LL



(d) Test results of WS62256LLPG-70

Fig. 12. Test results of NIST randomness test suite.

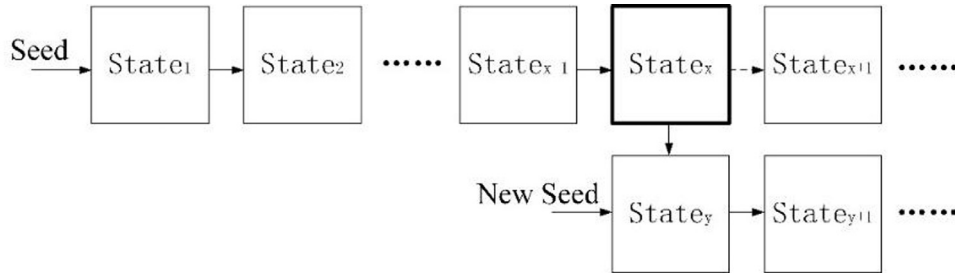


Fig. 13. Change of DRBG internal states.

SP 800–22 test suite. Comparing with previous PRNGs, the throughput of our CSPRNG may not be the fastest, but the reseed operation can insure the security of PRNG through the introduction of new entropy. On the other hand, our CSPRNG is the only PRNG that can test the entropy of entropy sources in real time. This resists the attacks aiming at entropy sources.

In order to comparison with the existing work, we converted the FPGA resource consumption of our implementation into the gate level. As shown in Table 2, due to the huge memory consumption of Get Entropy module that used to reseeding and entropy test, our implementation is the most resource-consumption and the throughput of our CSPRNG may not be the fastest, but the reseed operation can insure the security of PRNG through the introduction of new entropy. On the other hand, our CSPRNG is the only PRNG that can test the entropy of entropy sources online. This resists the attacks aiming at entropy sources.

6. Security discussion

Fig. 13 depicts the change of DRBG internal states during the CSPRNG system operation. The pseudo-random numbers are derived from these internal states upon request by an application. The following discussions focus on backtracking resistance and prediction resistance of our DRBG. Suppose that a state compromise occurs at $State_x$ that contains secret information.

Backtracking resistance means that a compromise of internal state has no effect on the security of prior outputs [3]. The generation algorithm in DRBG is hash algorithm which is a one-way function. The unidirectionality of one-way function can insure that the outputs from $State_1$ to $State_{x-1}$ cannot be distinguished from random output and these prior internal states also cannot be recovered, given knowledge of the secret information in $State_x$. Therefore, this DRBG can provide backtracking resistance.

Prediction resistance means that a compromise of internal state has no effect on the security of future outputs [3]. The introduction of fresh entropy via reseeding can hide the secret of $State_x$ in $State_y$. It makes the adversary not be able to compute $State_{x+1}$, simply by knowing the secret information in $State_x$. The periodical reseeding operation can insure that the future internal states cannot be predicted, given knowledge of $State_x$. Therefore, this DRBG can provide prediction resistance.

Furthermore, it is more reasonable to use CSPRNG in cryptographic protocols. CSPRNG requirements fall into two groups: first, that they can pass statistical randomness tests; and secondly, that they hold up well under serious attacks, even when part of their initial or running state becomes available to an attacker. These attacks are usually divided into three categories [28]: direct cryptanalytic attacks, side-channel attacks, and entropy source attacks.

6.1. Direct cryptanalytic attacks

For direct cryptanalytic attacks [29], an adversary appears to be required cryptanalysis of SHA-256. As far as we know, this hash

algorithm has not been cracked. Even if it was cracked in the future, SHA-256 can be replaced with SHA-3 to resist this kind of attacks.

6.2. Side-channel attacks

For side-channel attacks, there are two ways of making a state compromise: timing attacks [30] and power analysis attacks [31]. In this CSPRNG system, the length of seeds fed into DRBG is always the same, resulting in the same consumption during pseudo-random numbers generation. Therefore, it is hard for attackers to utilize the difference in time information to implement an attack. The power analysis attacks aim at the leakage of power consumption information during the operation of PRNG. So far, there is no published power attack on NIST SP800-90. Even if the attackers obtain the internal state of DRBG by the above-mentioned two approaches, the backtracking resistance and the prediction resistance can insure the unpredictability of outputs.

Furthermore, Differential Fault Analysis (DFA) [32] is a powerful method to attack a secret-key cryptosystems. It can be considered as a combination of cryptanalysis and SCAs. Therefore, it is considered as a very powerful attack even compared to other SCAs. Fig. 14 is a simplified model of DFA. It should be mentioned that this attack should be carried out under the assumption that the attacker need to run the device with the same plaintext P and the same key K in obtaining correct C (ciphertext) and C' (faulty ciphertext). And then, this attack can obtain K by P , I (correct intermediate value), I' (faulty intermediate value), C and C' .

However, this powerful attack may not be suitable for our scheme. The reason is that the output of SRAM PUFs is unstable due to the affection of noise, aging or even the faults induced by the attacker. Therefore, referring to the DFA attack with same plaintext P and K , it is hard to run the device with the same SRAM PUFs output (stored in FIFO). Furthermore, the fault in SRAM and FIFO induced by the attacker may affect the entropy of seed. And then, the randomness of the DRBG output can be affected by this attack. In our scheme, the Get Entropy module have the functions of monitoring the entropy of the seed. If the seed cannot meet the requirements of DRBG, our scheme will start a reseeding process.

6.3. Entropy source attacks

For entropy source attacks, an adversary needs to predict the entropy data onto the next moments. In this paper, so far, there has been

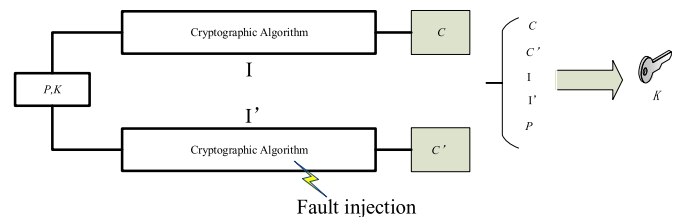


Fig. 14. The model of DFA.

two methods to crack SRAM PUFs: Semi-invasive attacks [33,34] and remanence decay based attacks [35].

Nedospasov et al. [33] is aimed at SRAM PUFs based authentication and key generation, which utilizes the stable output of 0-skewed and 1-skewed cells. On the contrary, the existence of neutral-skewed cells will increase the difficulty of such attack. Helfmeier and Boit [34] bias the states of cells to clone the unique responses from SRAM PUFs by modifying the circuits of SRAM. It also assumes that a fuzzy extractor algorithm is used to filter the neutral-skewed cells, which means this attack is also aimed at the stable output of SRAM PUFs. Our CSPRNG utilizes the neutral-skewed cells which belong to the probability models influenced by temperature, voltage, chip aging and other factors. The dynamically reseeding operation can make the neutral-skewed cells produce unpredictable output and introduce new entropy into system. Therefore, it is difficult to predict or clone the entropy data of SRAM PUFs in our CSPRNG.

Oren et al. [35] inject errors into SRAM PUFs by controlling the supply voltage or power-down time of SRAM chips. Through this method, an adversary can obtain the entire start-up pattern of SRAM PUFs and clone the initial states. However, the control process will dramatically change the $(H_{min})_{total}$ in SRAM PUFs (cf. Fig. 4). Our Get Entropy module can detect the change and return error signal.

In conclusion, this CSPRNG system conforms to the two requirements of CSPRNG. Therefore, we believe that this system can be securely used in cryptographic protocols.

7. Conclusion

In this paper we have presented a construction of CSPRNG system, which is a combination of a Get Entropy module used for verifying the availability of SRAM PUFs and extracting seeds from SRAM PUFs and a DRBG used for converting these seeds into pseudo-random numbers.

The Get Entropy module verifies the min-entropy of SRAM PUFs from SRAM start-up patterns in real time. After the verification, this module can compress entropy data into truly random seeds. Furthermore, the suggested values of module key parameters have been given by experiment results in Section 3. The results also show that all series of SRAM chips tested in this paper can be used as SRAM PUFs. Therefore, this module is applicable to a wide range of SRAM chips.

The DRBG is designed according to the specifications of NIST 800–90A. This module can utilize truly random seeds and deterministic algorithm to generate pseudo-random numbers. The periodic reseeding operation and the usage of hash algorithm insure that this DRBG can provide backtracking resistance and prediction resistance.

Combining the Get Entropy module with the DRBG, as described in Section 5, results in our CSPRNG system. The pseudo-random numbers generated by this system can pass all random tests of the NIST Statistical Test Suite and the throughput is up to 598.1 Mbps in our configuration. Furthermore, based on the discussion in Section 6 we can conclude that our implementation can be securely applied to cryptographic protocols.

Acknowledgment

This work has been supported by The National Natural Science Foundation of China (The method of information security research based on PUFs lossless compression) Nos. 61571116.

References

- [1] K.H. Tsoi, K.H. Leung, P.H.W. Leong, Compact FPGA-based true and pseudo random number generators, FCCM 2003, Napa, CA, 8–11 April, IEEE, New York, 2003, p. 51.

- [2] C. Wachsmann, A.R. Sadeghi, Physically unclonable functions (PUFs): applications, models, and future directions, *Synth. Lect. Inf. Secur. Privacy Trust* 9 (1) (2014) 1–91.
- [3] E.B. Barker, J.M. Kelsey, SP 800-90A. Recommendation for Random Number Generation Using Deterministic Random Bit Generators, National Institute of Standards & Technology, MD, USA, 2012.
- [4] R. Pappu, B. Recht, J. Taylor, N. Gershenfeld, Physical one-way functions, *Science* 297 (5589) (2002) 2026–2030.
- [5] J.W. Lee, D. Lim, B. Gassend, A technique to build a secret key in integrated circuits for identification and authentication applications, 2004 Symposium on VLSI Circuits, Honolulu, Hawaii, 17–19 June, IEEE, New York, 2004, pp. 176–179.
- [6] L. Lin, D. Holcomb, D.K. Krishnappa, P. Shabadi, Low-power sub-threshold design of secure physical unclonable functions, *International Symposium on Low Power Electronics and Design*, Austin, TX, 18–20 August, IEEE, New York, 2010, pp. 43–48.
- [7] B. Gassend, D. Clarke, M. Van Dijk, S. Devadas, Silicon physical random functions, *ACM Conference on Computer and Communications Security* 2002, Washington, DC, 17–21 November, ACM, New York, 2002, pp. 148–160.
- [8] A. Maiti, J. Casarona, L. Mchale, P. Schaumont, A large scale characterization of RO-PUF, *IEEE International Symposium on Hardware-Oriented Security and Trust*, CA, USA, 13–14 June, IEEE, New York, 2010, pp. 94–99.
- [9] J. Guajardo, S.S. Kumar, G.J. Schrijen, P. Tuyls, FPGA intrinsic PUFs and their use for IP protection, *Cryptographic Hardware and Embedded Systems (CHES 2007)*, Vienna, Austria, 10–13 September, Springer-Verlag, Berlin, 2007, pp. 63–80.
- [10] R. Maes, P. Tuyls, I. Verbauwhede, Intrinsic PUFs from flip-flops on reconfigurable devices, *Workshop on Information and System Security (WISec 2008)*, Eindhoven, NL, 13–14 November, 2008, p. 17.
- [11] S.S. Kumar, J. Guajardo, R. Maes, G.J. Schrijen, P. Tuyls, Extended abstract: the butterfly PUF protecting IP on every FPGA, *IEEE International Workshop on Hardware-Oriented Security and Trust*, CA, USA, 9–9 June, IEEE, New York, 2008, pp. 67–70.
- [12] D.E. Holcomb, W.P. Burleson, K. Fu, Power-Up SRAM state as an identifying fingerprint and source of true random numbers, *IEEE Trans. Comput.* 58 (9) (2009) 1198–1210.
- [13] V.V.D. Leest, E.V.D. Sluis, G.J. Schrijen, P. Tuyls, H. Handschuh, Efficient implementation of true random number generator based on SRAM PUFs, *Cryptography Secur* 6805 (2012) 300–318.
- [14] D. Li, Z. Lu, X. Zou, Z. Liu, PUFKEY: a high-security and high-throughput hardware true random number generator for sensor networks, *Sensors* 15 (10) (2015) 26251–26266.
- [15] J.C. Cerda, C.D. Martinez, J.M. Comer, D.H.K. Hoe, An efficient FPGA random number generator using LFSRs and cellular automata, *Midwest Sympos. Circuits Syst.* 327 (4) (2012) 912–915.
- [16] Y. Liu, K. Wu, R. Karri, Scan-based attacks on linear feedback shift register based stream ciphers, *ACM Trans. Design Autom. Electron. Syst.* 16 (2) (2011) 1–15.
- [17] J.M. Rabaey, *Digital Integrated Circuits: A Design Perspective*, Prentice Hall, NJ, USA, 2003.
- [18] A. Van Herrewwege, *Lightweight PUF-based Key and Random Number Generation*, KU Leuven, Belgium, 2015.
- [19] E.B. Barker, J.M. Kelsey, SP 800-90B Recommendation for the Entropy Sources Used for Random Bit Generation, National Institute of Standards & Technology, MD, USA, 2012.
- [20] C.C. Fd, S. Skorobogatov, Low Temperature Data Remanence in Static RAM, *University of Cambridge Computer Laboratory*, Cambridge, 2002.
- [21] NIST FIPS 180-4, Secure Hash Standard (SHS), 2015.
- [22] http://opencores.org/project/sha_core.
- [23] L.E. Bassham III, A.L. Rukhin, et al., SP 800-22 rev. 1a. A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications, National Institute of Standards & Technology, MD, USA, 2010.
- [24] C.Y. Li, Y.H. Chen, T.Y. Chang, L.Y. Deng, K. To, Period extension and randomness enhancement using high-throughput reseeding-mixing PRNG, *IEEE Trans. Large Scale Integr. Syst.* 20 (2) (2012) 385–389.
- [25] T. Addabbo, M. Alioto, A. Fort, A. Pasini, S. Rocchi, V. Vignoli, A class of maximum-period nonlinear congruential generators derived from the Rényi chaotic map, *IEEE Trans. Circuits Syst. I* 54 (4) (2007) 816–828.
- [26] C.Y. Li, T.Y. Chang, C.C. Huang, A nonlinear PRNG using digitized logistic map with self-reseeding method, 2010 International Symposium on VLSI Design Automation and Test (VLSI-DAT), 26–29 April, Hsin Chu, Taiwan, IEEE, New York, 2010, pp. 108–111.
- [27] B.M. Gammel, R. Gottfert, O. Kniffler, An NLFSR-based stream cipher, 2006 IEEE International Symposium on Circuits and Systems, Island of Kos, Greece, 21–24 May, 4, IEEE, New York, 2006, p. 2920.
- [28] J. Kelsey, B. Schneier, N. Ferguson, Yarrow-160: Notes on the Design and Analysis of the Yarrow Cryptographic Pseudorandom Number Generator, Springer-Verlag, Berlin, 1999.
- [29] J. Kelsey, B. Schneier, D. Wagner, C. Hall, Cryptanalytic attacks on pseudorandom number generators, *Lect. Notes Comput. Sci.* 1372 (2000) 168–188.
- [30] P.C. Kocher, Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems, *Advances in Cryptology-CRYPTO '96*, Springer-Verlag, Berlin, 1996, pp. 104–113.

- [31] P. Kocher, J. Jaffe, B. Jun, Differential power analysis, *Advances in Cryptology-CRYPTO' 99*, Springer-Verlag, Berlin, 1999, pp. 388–397.
- [32] P. Dusart, G. Letourneux, O. Vivolo, Differential fault analysis on A.E.S. *Applied Cryptography and Network Security*, 2003:293–306. Springer Berlin Heidelberg.
- [33] D. Nedospasov, J.P. Seifert, C. Helfmeier, C. Boit, Invasive PUF analysis, *The Workshop on Fault Diagnosis & Tolerance in Cryptography*, CA, USA, 20–20 August, IEEE, New York, 2013, pp. 30–38.
- [34] C. Helfmeier, C. Boit, Cloning physically unclonable functions, *IEEE International Symposium on Hardware-Oriented Security and Trust*, Austin, TX, 2–3 June, IEEE, New York, 2013, pp. 1–6.
- [35] Y. Oren, A.R. Sadeghi, C. Wachsmann, On the effectiveness of the remanence decay side-channel to clone memory-based PUFs, *Cryptographic Hardware and Embedded Systems - CHES 2013*, Springer-Verlag, Berlin, 2013, pp. 107–125.



Shuai Chen received the master's degree in integrated circuit engineering from the Southeast University, Nanjing, China, in 2016. He is currently pursuing the Ph.D. degrees from the Southeast University, Nanjing, China.

He published several papers in the core journals at home and abroad. His current research interests include the information security and digital circuit design.



Bing Li received the Ph.D. degree from the Southeast University, Nanjing, China, in 2004.

He is a Professor at the School of Microelectronics, Southeast University, where he leads the advanced cloud system joint research team. His current research interests include the design of safety information exchange system and circuit system.



Cenjun Zhou is currently pursuing the master's degree in integrated circuit engineering from the Southeast University, Nanjing, China.

His current research interests include the information security and digital circuit design.